

GUIA DE ALINHAMENTO E EXCELÊNCIA TÉCNICA

Projeto Crochettei

Material de Apoio Interno para Desenvolvimento Web

ALICE LOURENÇO, DAVI SUTIL E KARYNA CARBAS

SERRA, 2026

1. Introdução Estratégica do Projeto

O Projeto Crochettei não nasce apenas como um marketplace, mas como o elo tecnológico essencial para um setor que é um gigante econômico no Espírito Santo: o artesanato. Com uma movimentação anual estimada em R\$2 bilhões no estado, o setor esbarra em um abismo estrutural: enquanto 15.982 artesãos possuem registro oficial, estima-se que cerca de 40 mil profissionais operam na informalidade, à margem de crédito, garantias previdenciárias e canais digitais de venda.

No centro desta realidade, encontramos a nossa persona: mulheres, frequentemente na faixa dos 40 a 60 anos, chefes de família que residem no Espírito Santo e dependem do artesanato como fonte de renda essencial. Essas profissionais são talentosas e resilientes, mas enfrentam barreiras digitais severas e uma rotina exaustiva de gestão, muitas vezes sacrificando a produção de suas peças para lidar com a complexidade de múltiplos canais de venda.

Nossa missão é clara: usar a tecnologia para eliminar as barreiras que impedem o crescimento desses profissionais. Mais de 70% da força de trabalho do setor enfrenta a informalidade não por falta de demanda, mas pelo medo da burocracia e da complexidade tecnológica. O Crochettei atua como um "escudo digital". Através de uma interface intuitiva, estruturada em uma jornada de apenas 5 cliques, transformamos o medo em oportunidade.

Queremos que o artesanato ultrapasse a barreira da comercialização limitada a feiras e encontros presenciais. Mais do que vender, queremos simplificar o dia a dia do artesão: em vez de se perder em múltiplas redes sociais e meios de comunicação, ele terá um ambiente de trabalho dedicado e formalizado. O Crochettei rompe o teto de faturamento tradicional, permitindo que a produção circule 24/7, garantindo que a artesã possa focar na qualidade de sua obra enquanto nossa plataforma cuida da escala e da visibilidade em todo o estado.

1.1 Definição do Mínimo Produto Viável (MVP)

Para validar o nosso propósito, e demonstrar que mais de um projeto idealista é algo realista, a elaboração do Produto Mínimo Viável (MVP), é crucial, é aqui que provamos que nossas urgências, aquilo que torna nossa plataforma utilizável e eficaz, vai de encontro com a dor da nossa persona. Nosso foco está em garantir uma jornada de alta conversão e usabilidade extrema, onde a barreira de entrada para a artesã seja limitada a **5 cliques**, enquanto o fluxo do comprador segue padrões de mercado intuitivos e eficientes.

O núcleo vital (core) do nosso MVP é composto pelas seguintes funcionalidades:

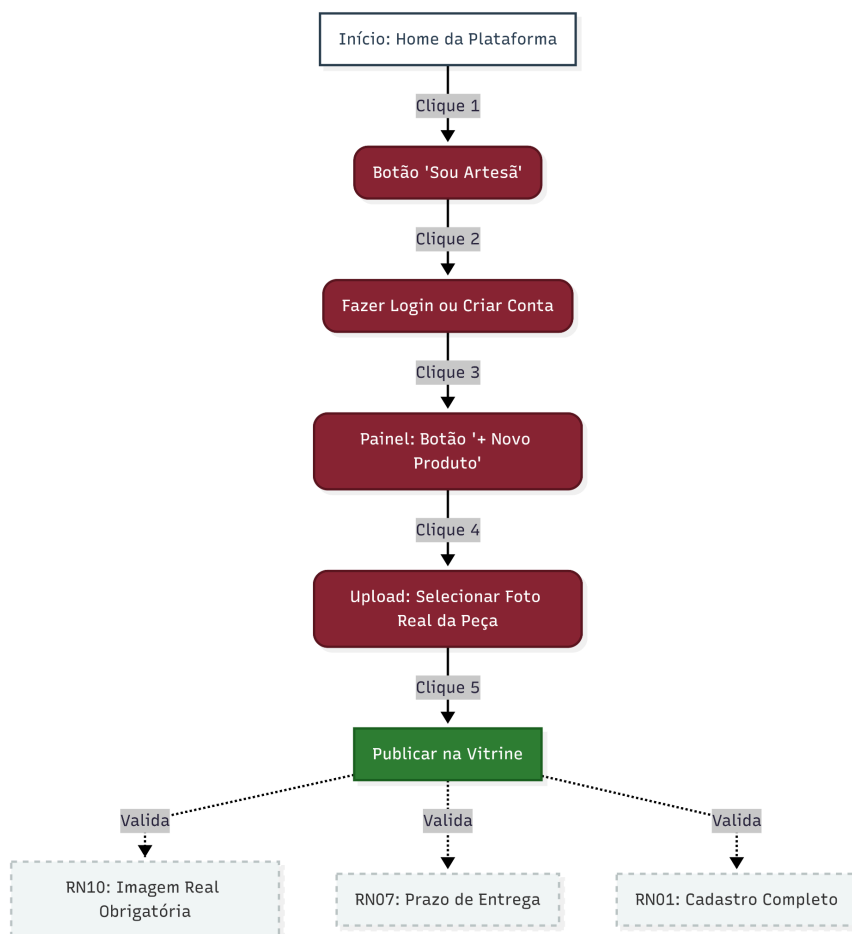
- **Módulos da Artesã (Inclusão Digital):** Interface simplificada para gestão de perfil e catálogo. Para garantir a qualidade e a conformidade, aplicamos as seguintes Regras de Negócio (RN):
 - **RN01:** Cadastro completo como pré-requisito para ativação de produtos.
 - **RN07:** Obrigatoriedade de prazo de entrega/fabricação por item.
 - **RN10:** Obrigatoriedade de imagens reais para autenticidade da peça.
- **Vitrine e Experiência de Compra:** Navegação intuitiva com filtros básicos por categoria e fluxo de carrinho descomplicado para o consumidor final.

- **Checkout e Logística:** Processo de fechamento de pedido estruturado com validação obrigatória de dados (CPF e Endereço). O fluxo integra pagamento e gestão de status de entrega, permitindo que a artesã monitore cada etapa de forma manual, garantindo segurança na transação sem a complexidade de APIs externas em um primeiro momento.

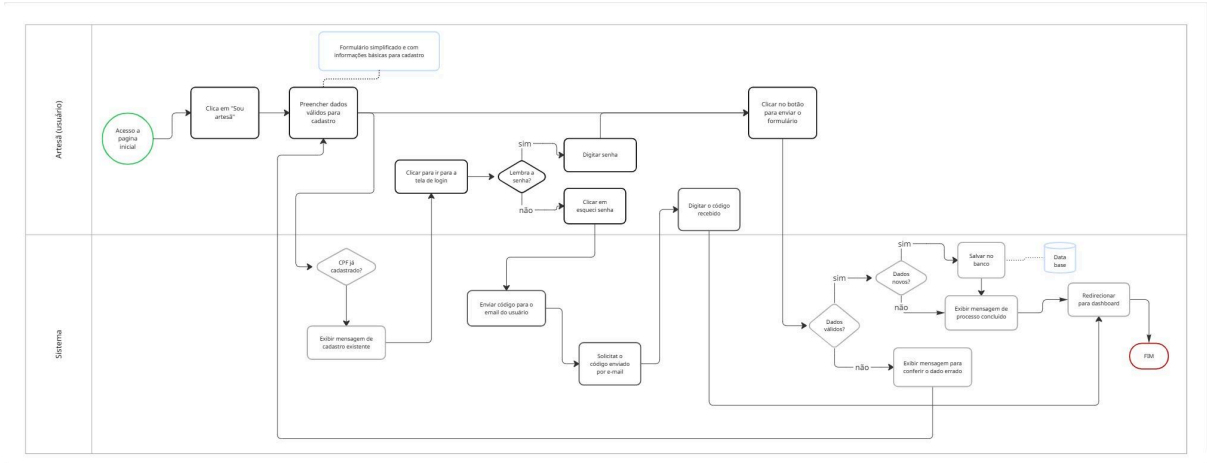
Funcionalidades como relatórios financeiros avançados, sistemas de avaliação (reviews), e automação de fretes e rastreio via API são essenciais, mas pertencem a um momento de escala. A adoção da **Clean Architecture** nos permite postergar esses desenvolvimentos sem comprometer a integridade do nosso núcleo. Dessa forma, mantemos o projeto ágil, sustentável e focado no que gera valor imediato: a venda da artesã e a sua transição para o empreendedorismo formal.

1.1.1 Diagrama de Fluxo usabilidade Estrita: A jornada de 5 cliques da Artesã

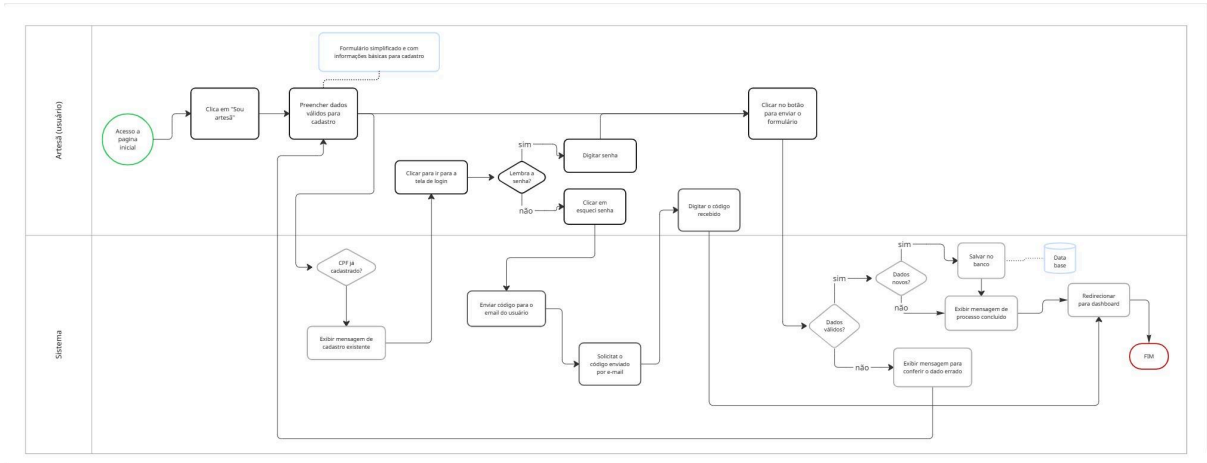
Fluxo Geral do sistema



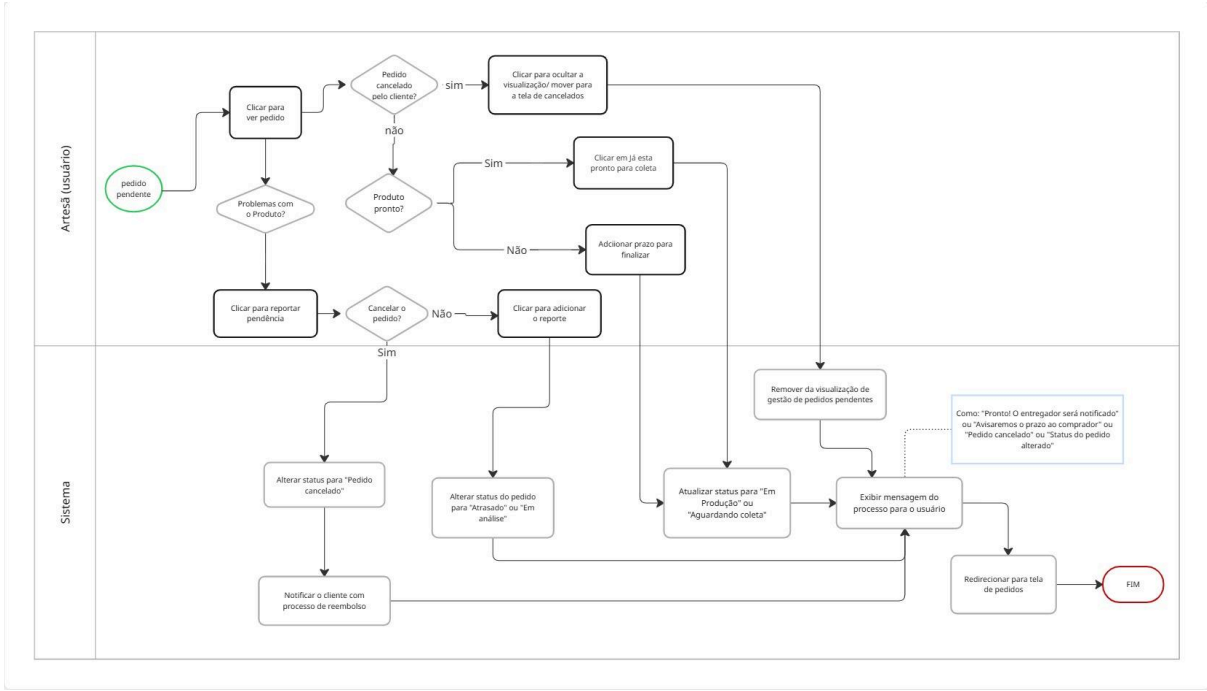
1.1.2 Fluxo de acesso ao sistema (como artesã)



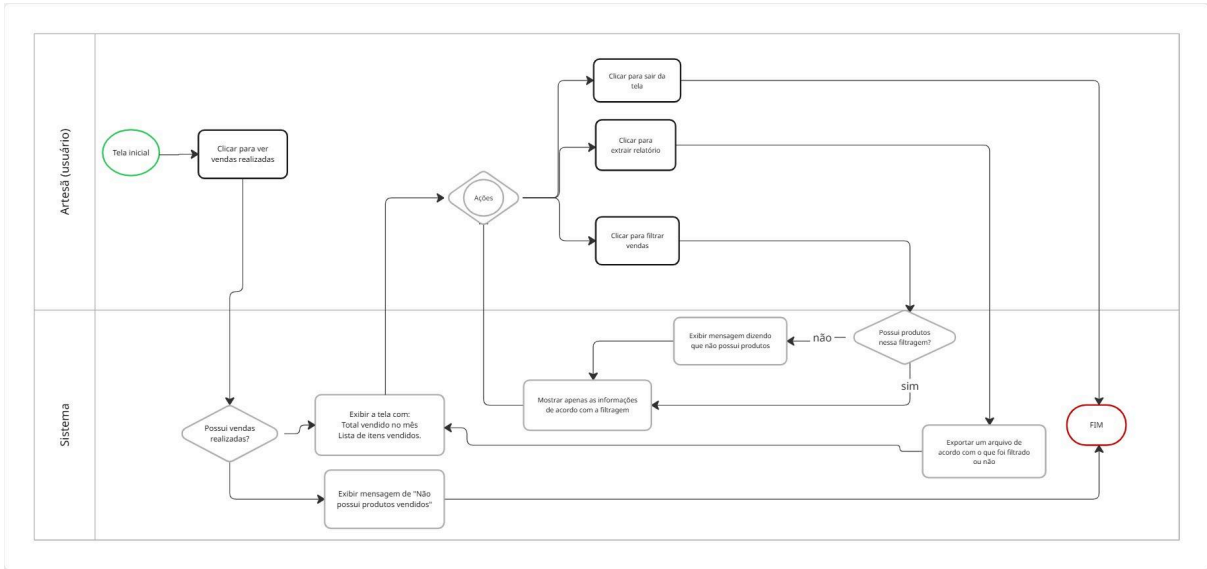
1.1.3 Fluxo de cadastro de produto



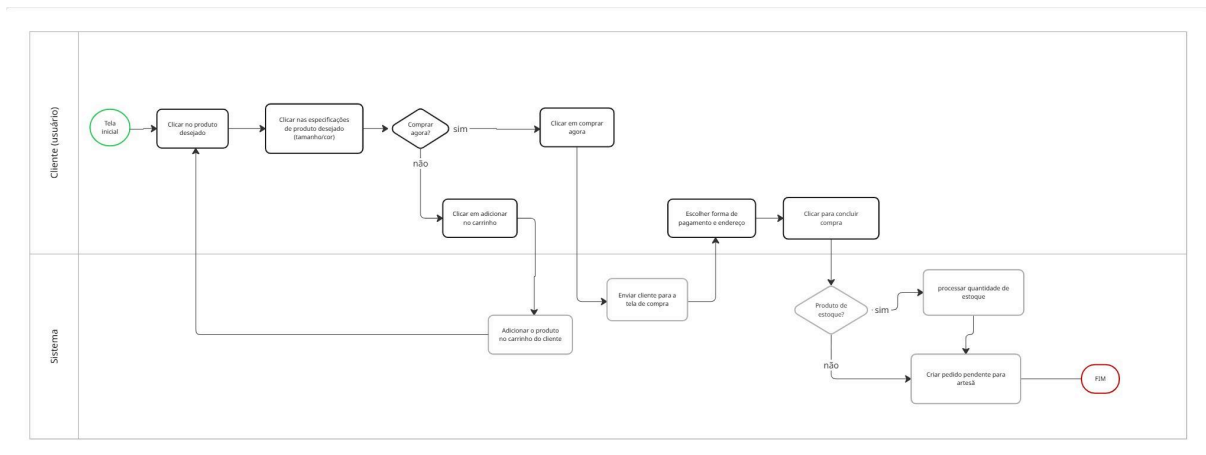
1.1.4 Fluxo de gestão de vendas



1.1.5 Fluxo de relatório de vendas



1.1.6 Fluxo de compra do produto



1.2 MoSCoW:

Para guiar o desenvolvimento do projeto e garantir que o foco permaneça nas dores reais das nossas artesãs, adotamos a metodologia MoSCoW para o mapeamento e priorização de funcionalidades. Essa ferramenta foi fundamental para delimitar o escopo do nosso MVP, permitindo que direcionássemos nossos esforços para o que é, de fato, o motor da nossa plataforma.

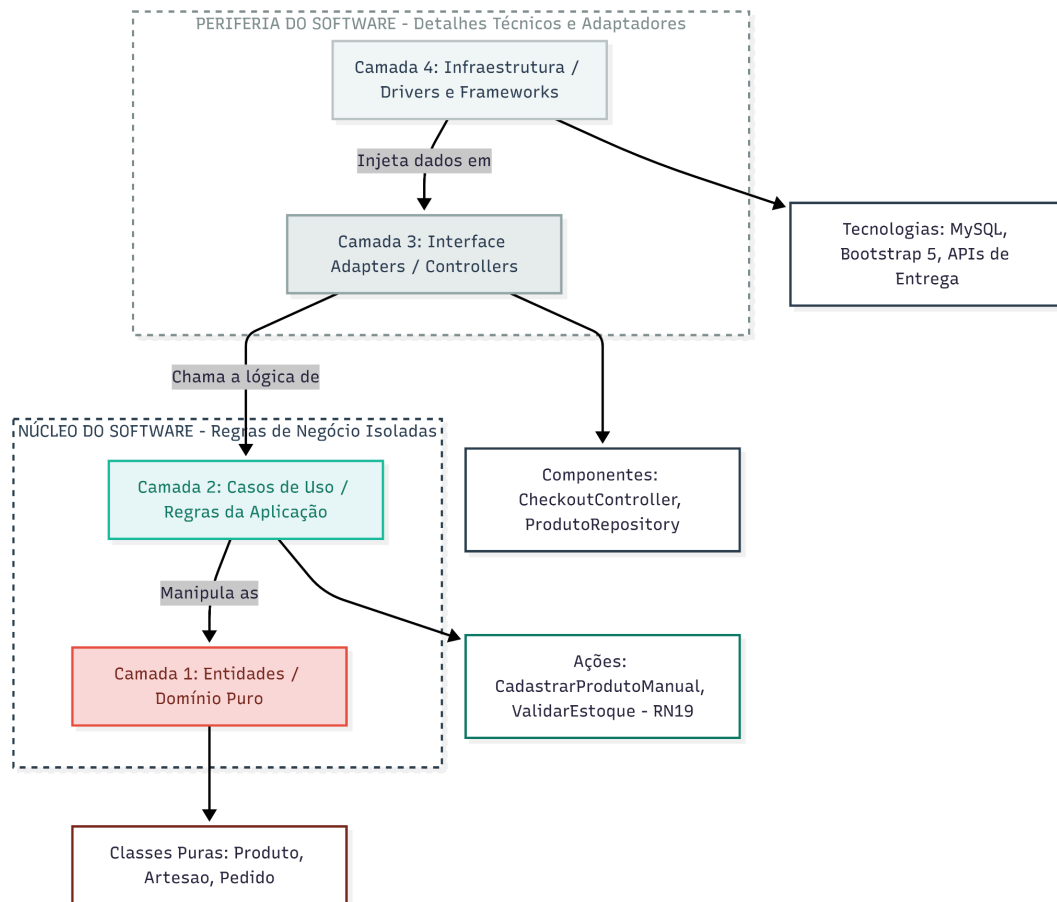
As funcionalidades classificadas como Must have compõem os pilares inegociáveis do Crochettei. Sem o cadastro de artesãs e clientes, a gestão de produtos, o gerenciamento de pagamentos e a jornada completa de busca, carrinho e conclusão de pedidos, o ciclo básico de venda e compra seria impossível de ser executado. Priorizamos esses itens porque eles representam a entrega de valor imediata para a nossa persona, garantindo que o sistema seja utilizável e eficaz desde o primeiro dia.

Em um segundo nível, os requisitos Should have (como controle de estoque, edição de produtos e recuperação de senha) fornecem a autonomia necessária para a gestão diária da artesã, enquanto os itens Could have (como avaliações e notificações) ficam reservados para etapas futuras, visando a retenção e o feedback contínuo. Por fim, definimos que, neste momento, Won't have funcionalidades como integração complexa de APIs logísticas ou chats em tempo real. Essa é uma decisão estratégica deliberada: ao substituir automações complexas por processos manuais de validação, mantemos o custo de desenvolvimento otimizado e o foco total na solidez da nossa estrutura de base.

Essa priorização não apenas baliza o que será construído, mas fundamenta diretamente a escolha pela nossa abordagem técnica. É justamente para sustentar esse núcleo vital que adotamos um design de sistemas rigoroso e desacoplado, o qual exploraremos a seguir em nossa arquitetura.

Prioridade	Requisitos (Funcionalidades)	Justificativa
Must have	Cadastro de artesãs, Cadastro de clientes, Cadastro de produtos, Gerenciamento de pagamentos, Login e autenticação, Adição ao carrinho, Conclusão de pedidos.	São os pilares fundamentais. Sem esses, a plataforma não executa o ciclo básico de venda e compra.
Should have	Controle de estoque, Acompanhamento de vendas, Edição/exclusão de produtos, Recuperação de senha, Busca e filtro de produtos.	Essenciais para a gestão da artesã no dia a dia, garantindo autonomia sobre o que ela anuncia.
Could have	Avaliação de produtos/artesãs, Visualização de histórico de compras, Envio de notificações.	Melhoram a retenção e o feedback, mas não impedem o funcionamento da venda no MVP.
Won't have	Gerenciamento complexo de rotas de entrega (Integração de APIs), Cálculos dinâmicos de frete, Algoritmos de logística, Chat em tempo real	Foco em desenvolvimento web: Substituímos por lógica manual e processamento interno para focar na arquitetura MVC e na jornada da artesã.

1.2.1 Mapeamento de camadas da Clean Architecture aplicada ao mvp do Crochettei:



A espinha dorsal do projeto Crochettei é fundamentada nos princípios da *Clean Architecture*, uma abordagem estratégica que garante que o "coração" da nossa plataforma permaneça isolado e imune às constantes mudanças tecnológicas. Conforme ilustrado em nossa estrutura arquitetural, dividimos o sistema em dois domínios distintos: o Núcleo do Software e a Periferia. O Núcleo é o nosso ativo mais valioso, composto pela Camada 1, que abriga o Domínio Puro com as nossas classes essenciais (Produto, Artesão e Pedido) e pela Camada 2, onde residem os Casos de Uso, como o cadastro manual de produtos e a validação de estoque (RN19). Esta camada orquestra o fluxo de dados garantindo que as regras da aplicação sejam respeitadas, sem que precise saber qual banco de dados usamos ou qual interface o usuário visualiza.

Ao redor deste núcleo, posicionamos a Periferia do Software, que atua como uma blindagem técnica. A Camada 3 funciona como uma zona de adaptadores e controladores, sendo responsável por traduzir as requisições externas para o formato que o nosso núcleo compreende, utilizando componentes como o *CheckoutController* e o *ProdutoRepository*. Por fim, a Camada 4 é onde residem os detalhes de implementação e infraestrutura, como o banco de dados MySQL, o framework Bootstrap 5 e as futuras integrações de APIs de entrega.

Essa separação de preocupações é o que torna o Crochettei um projeto tecnicamente sustentável e economicamente inteligente. Ao isolar a complexidade técnica, reduzimos

drasticamente os custos de desenvolvimento e manutenção a longo prazo, pois alterações na interface ou trocas de fornecedores tecnológicos não exigem a reescrita das regras de negócio. Essa eficiência financeira casa perfeitamente com o propósito do Crochettei: como nossa estrutura é enxuta e focada nas dificuldades reais de uma persona que enfrenta barreiras tecnológicas, evitamos o "inchaço" de funcionalidades desnecessárias que encareceriam a plataforma.

Dessa forma, garantimos que cada real investido no projeto seja revertido em usabilidade para a artesã, permitindo que o sistema seja acessível, estável e lucrativo desde o MVP. Em suma, nosso design arquitetural não é apenas uma escolha técnica; é um compromisso social que garante que a tecnologia sirva como uma alavanca invisível, robusta e economicamente viável para profissionalizar o artesanato capixaba.

Mapeamento das Camadas (Crochettei)

Camada	O que é no Crochetei?	Responsabilidade
Entities (Entidades)	Os Models (pasta app/Models/). Apesar de conectarem com o banco no CodeIgniter, é dentro deles que definimos as validações de integridade, como os tamanhos de string e a regra greater_than[0] no ProdutoModel para impedir preços negativos.	Contêm os dados e as regras mais puras (ex: "um pedido não pode ter valor negativo"). Não acessam banco de dados.
Use Cases (Casos de Uso)	As classes dentro da pasta app/UseCases/ (ex: CheckoutUseCase.php, GerenciarCarrinhoUseCase.php). Elas puxam a lógica isolada para não misturar com requisições web.	Onde a lógica acontece. Eles orquestram: "Recebo o produto, valido as regras, salvo no repositório".
Interface Adapters	Os Controllers (app/Controllers/) e as Views (app/Views/). Eles pegam as informações da interface visual, limpam os POSTs e enviam as requisições traduzidas para os Casos de Uso.	O "tradutor". Recebe a requisição HTTP (do form da artesã), chama o Use Case e decide qual View retornar.
Frameworks & Drivers	MySQL, Bootstrap 5, PHP Framework.	A parte externa. É o que "roda" o sistema. Se você mudar de MySQL para PostgreSQL, só altera aqui.

2. Detalhamento Técnico Orientado à Avaliação

2.1 Arquitetura MVC & Segurança de Dados

A arquitetura do **Crochettei** foi projetada sob o princípio da **Defesa em Profundidade**. Para garantir a integridade dos dados das artesãs e a segurança transacional dos clientes, a plataforma utiliza de forma simbiótica os recursos nativos do framework **CodeIgniter 4** e o isolamento rígido promovido pelo padrão arquitetural **MVC** (Model-View-Controller).

O fundamento central dessa escolha é a **separação estrita de responsabilidades**: estabelecemos uma premissa de desconfiança sistemática em relação a qualquer entrada do usuário (proveniente da *View*), ao mesmo tempo em que blindamos o acesso ao banco de dados (restrito ao *Model*), garantindo que a interface jamais interaja diretamente com a camada de persistência.

Abaixo, detalha-se como as escolhas arquiteturais justificam e sustentam a segurança do ecossistema contra os principais vetores de ataque da Web (OWASP Top 10).

2.1.1 Controller: A barreira de entrada e controle de escopo

O Controller atua como o "segurança da porta" da aplicação. Justificamos a centralização das regras de validação nesta camada para impedir que requisições maliciosas ou malformadas consumam processamento do servidor ou alcancem o banco de dados.

- **Autenticação Coesa e Filtros de Sessão:** Através de *Filters* acoplados às rotas de controllers críticos como `ArtesaController` e `CheckoutController`, o sistema valida o estado das Sessões HTTP em tempo de roteamento. Se o usuário não possuir credenciais válidas, o acesso é negado antes mesmo da Controller ser instanciada.
- **Proteção Ativa contra CSRF (Cross-Site Request Forgery):** O ecossistema exige e valida tokens criptográficos de uso único para qualquer requisição de alteração de estado (POST, PUT, DELETE). Isso inviabiliza ataques onde agentes externos tentam forjar ações (como disparar compras ou excluir produtos) em nome de um usuário autenticado.
- **Validação Estrita de Inputs:** Adotamos uma política de *Whitelisting* (lista de permissões) no recebimento de dados. Formatos de CPF, tamanho de senhas e tipos de arquivos de imagem são sanitizados e validados imediatamente na entrada, bloqueando anomalias na origem.
- **Mitigação de IDOR (Insecure Direct Object Reference) via Multitenancy:** Uma das maiores vulnerabilidades em marketplaces é a manipulação de parâmetros (ex: alterar o ID de um produto na URL para tentar excluir o item de outra artesã). Justificamos a amarração de escopo forçando o sistema a

cruzar o ID do recurso solicitado com o `artesaoid` extraído estritamente da sessão segura do servidor, e nunca do formulário ou da URL.

2.1.2 Model: O encapsulamento e a blindagem do banco de dados

O *Model* é a única camada que conhece a topologia do banco de dados. Isolamos completamente essa comunicação para criar um perímetro de segurança em torno do ativo mais valioso da plataforma: a informação.

- **Prevenção Absoluta contra SQL Injection (SQLi):** Justifica-se o uso mandatório do *Query Builder* nativo do CodeIgniter 4 e de *Prepared Statements* (declarações preparadas). Ao separar a estrutura da query dos dados enviados pelo usuário, qualquer tentativa de injeção de código SQL malicioso (ex: ' OR 1=1; DROP TABLE...') é neutralizada pelo SGBD, que passa a tratar o comando estritamente como uma string literal inofensiva.
- **Criptografia de Estado da Arte para Credenciais:** O armazenamento de senhas em texto claro é terminantemente proibido. O Model faz uso do algoritmo Bcrypt por meio da função `password_hash()`, que adiciona uma camada de *salting* (dados aleatórios) a cada registro. Essa escolha garante que, mesmo na hipótese de um vazamento físico do banco de dados, o processo de engenharia reversa das senhas seja computacionalmente inviável.
- **Proteção contra Mass Assignment (Injeção de Propriedades):** Para evitar que um atacante manipule requisições HTTP adicionando colunas protegidas (como tentar injetar `is_admin = 1` no fluxo de cadastro), configuramos rigidamente a propriedade `allowedFields` em nossos modelos. O CodeIgniter descarta sumariamente qualquer campo que não tenha sido explicitamente autorizado para preenchimento em massa.

2.1.3 View: O visor protegido e a camada de saída

View limita-se estritamente à renderização da interface. Por premissa arquitetural, ela é agnóstica em relação às regras de negócio e ao banco de dados. Sua responsabilidade de segurança concentra-se na integridade do que é exibido no navegador do cliente.

- **Neutralização de XSS (Cross-Site Scripting):** Para evitar que códigos maliciosos injetados por terceiros sejam executados no navegador de outros usuários (como scripts que roubam cookies de sessão), toda e qualquer variável dinâmica impressa na tela é tratada pela função nativa `esc()`. O sistema converte caracteres especiais de HTML em suas respectivas entidades de texto (transformando `<script>` em `<script>`), garantindo uma exibição segura e inofensiva.
- **Injeção Segura de Tokens:** As Views dos formulários incorporam dinamicamente a diretiva `<?= csrf_field() ?>`. Essa escolha de design simplifica o desenvolvimento e garante que nenhum formulário seja

construído na interface sem a contraparte de segurança exigida pela Controller.

A escolha pelo padrão **MVC** associado ao **CodeIgniter 4** não foi apenas estética, mas uma decisão estratégica de segurança. Ao estabelecer **fronteiras claras de isolamento e baixo acoplamento entre os componentes**, mitigamos o erro humano e limitamos o raio de exposição a falhas: a View não possui privilégios de acesso e limita-se à apresentação; ela reporta estritamente à Controller, que atua sob o princípio do privilégio mínimo, inspecionando e higienizando os dados na entrada. Por fim, o Model manipula o banco de dados utilizando mecanismos de parametrização (Prepared Statements). Essa engrenagem de responsabilidades bem delimitadas garante que o **Crochettei** seja uma plataforma resiliente, confiável e em total conformidade com as melhores práticas de engenharia de software do mercado.

2.2 Regras de Negócio Transpostas para Validação de Código

ID RN	Regras de Negócio (extraídas da matéria de PS)	Contexto de Negócio (nossa justificativa para essa regra)	Implementação Técnica (Back-end) (como vamos implementar, qual tecnologia e etc.)
RN05	Os artesãos só são notificados da venda após a confirmação de pagamento.	Evita que o artesão inicie a produção de uma peça personalizada ou reserve um produto em estoque sem a garantia financeira, prevenindo prejuízos com a matéria-prima e tempo	Criação de um endpoint de webhook no PHP/codeigniter para receber o payload assíncrono do gateway de pagamento. Após o status mudar para “pago”, o sistema dispara o evento que envia notificações (e-mail, whatsapp, via API como Twilio)
RN06	Após 7 dias do prazo de envio o cliente deve poder cancelar o pedido e ter reembolso integral	Protege o consumidor contra atrasos abusivos ou abandono por parte do artesão, garantindo a confiabilidade e o direito de	Lógica na Controller de Pedidos que valida se <code>data_atual > (data_limite_envio + 7 dias)</code> . Se verdadeiro, libera o botão de cancelamento para o cliente e realiza uma chamada via

		arrependimento respaldado pela plataforma.	API Rest (POST /refunds) do Gateway para estorno de 100% do valor no cartão/PIX.
RN14	Após 24 horas sem a confirmação do pagamento o pedido deve ser cancelado automaticamente	Liberar rapidamente os produtos de “Pronta Entrega” de volta ao estoque e desfaz reservas da agenda de produção, impedindo que carrinhos abandonados travem vendas.	Configuração de um Cron Job (tarefa agendada) no servidor linux para rodar de hora em hora. O script executará uma Query: <code>WHERE status_pagamento = 'pendente' AND created_at < NOW() - INTERVAL 24 HOUR</code> . O sistema cancela o pedido e executa uma Transaction para devolver os itens ao estoque.
RN18	O frete deve ser dividido entre as três partes, plataforma, artesão, e cliente.	Estratégia de precificação competitiva para baratear o custo de envio para o cliente final, atraindo mais vendas enquanto dilui os custos logísticos entre os envolvidos.	Integração com a API de logística (Melhor Envio/Correios) no Checkout para calcular o frete total. No backend, aplica-se a regra matemática de divisão e envia-se uma requisição de Split de Pagamento para o Gateway, roteando os valores líquidos automaticamente para as contas bancárias da artesã e da plataforma.

3. Experiência do Usuário (UI/UX) Adaptada à Persona

O design de interface e a jornada do usuário no **Crochettei** foram inteiramente moldados sob os princípios do **Design Inclusivo** e da **Acessibilidade Cognitiva**. A nossa persona central - mulheres de 40 a 60 anos, chefes de família no Espírito Santo, que dependem do artesanato, mas enfrentam severas barreiras digitais e sobrecarga de tempo - exige um sistema que reduza drasticamente a **carga cognitiva**.

A tecnologia não deve ser mais um fator de estresse em sua rotina exaustiva, mas sim um "escudo digital". Para transformar a complexidade tecnológica em oportunidade, implementamos as seguintes escolhas de UX/UI:

3.1 Acessibilidade visual e redução de atrito motor

Considerando a faixa etária da persona, que frequentemente lida com fadiga visual (presbiopia) ou falta de destreza com telas de toque pequenas, a interface foi adaptada fisicamente:

- **Elementos de UI Ampliados:** Utilizamos botões e alvos de clique consideravelmente maiores que os padrões convencionais de mercado, minimizando erros de digitação e toques acidentais.
- **Alto Contraste:** Aplicamos taxas estritas de contraste entre fontes, ícones e planos de fundo, garantindo legibilidade perfeita mesmo em ambientes com iluminação precária ou em telas de dispositivos mais antigos.

3.2 Arquitetura de informação e simplificação de fluxos (a regra dos 5 cliques)

Para combater o medo da burocracia e a exaustão da gestão diária, redesenhamos a arquitetura do sistema para encurtar caminhos:

- **Jornadas de Cliques Curtas:** Estruturamos os fluxos principais (como a listagem de uma venda) para serem concluídos em, no máximo, 5 cliques.
- **Segregação de Complexidade:** Dividimos o sistema em duas esferas claras. Fluxos operacionais simples e diários (como *Meus Pedidos* e *Meus Produtos*) são destacados e isolados visualmente de fluxos administrativos complexos (como configurações bancárias e de repasses), evitando que a artesã se sinta sobrecarregada ao abrir o painel.

3.3 Padrão de design “Wizard” para processos longos

O cadastro de um produto artesanal envolve muitas variáveis (fotos, preços, prazos). Para evitar o abandono de tela e a sensação de complexidade, aplicamos o padrão de design **Wizard (Passo a Passo)** no cadastro de produtos.

- Esse método quebra um formulário longo e intimidador em **micro etapas sequenciais**. A artesã foca em apenas uma pergunta por vez (ex: Passo 1: Foto; Passo 2: Nome), avançando de forma fluida e sem ansiedade até a conclusão.

3.4 Linguagem natural, familiaridade e inclusão por áudio (IA)

A barreira digital muitas vezes se manifesta no medo de errar termos técnicos ou na dificuldade de digitar longos textos descritivos. Mitigamos isso de três formas:

- **Linguagem Coloquial:** Eliminamos completamente o "corporativês" e termos técnicos de sistemas (como *SKU*, *Payload*, *Gateway*). A interface conversa com a artesã de forma humana, simples e acolhedora.
- **Transcrição de Áudio para Mitigação de Barreiras de Letramento:** Reconhecendo que muitas artesãs expressam o valor e as nuances de sua arte com maior fluidez por meio da oralidade, implementamos um recurso de transcrição de áudio. A usuária pode ditar livremente a descrição do produto

e suas especificações técnicas (como medidas e materiais); o sistema processa a fala e a converte automaticamente em texto formatado para os campos correspondentes. Essa escolha de design de IHC (Interação Homem-Computador) é um pilar de inclusão social, pois neutraliza de forma prática as barreiras do analfabetismo funcional e digital, garantindo autonomia plena à artesã, independentemente de seu nível de letramento.

3.5 Integração com canais de alta afinidade (WhatsApp)

Em vez de forçar a usuária a aprender a navegar por sistemas internos complexos de chat ou abertura de chamados, o **Crochettei** canaliza o suporte e a comunicação direta com os compradores para o **WhatsApp**.

- Por ser uma ferramenta que a persona já utiliza diariamente em sua vida pessoal e que possui curva de aprendizado zero, o uso do WhatsApp como hub de relacionamento humaniza o atendimento e traz segurança imediata para a gestão do negócio

Ao unificar IA de voz, interfaces de alto contraste, o método Wizard e a familiaridade do WhatsApp, o **Crochettei** cumpre seu papel de design inclusivo. A plataforma remove o peso da burocracia das costas da artesã capixaba, permitindo que ela foque no que faz de melhor: a sua arte.